

# ETL Test Console

QA Team Reference Guide — AI-assisted test authoring, mapping setup, and OneDrive team sharing

This guide covers five things: turning a Jira story into draft test cases, converting drafted or existing test cases into runnable suites and scenarios, where to put mapping documents, and setting up shared OneDrive run history across the team.

*All commands below assume the project root is **C:\etl\_testing** — adjust if your VM uses a different path.*

## SECTION 1

### Jira Story → Draft Test Cases

Uses the **jira-to-test-cases** Cursor skill. Turns a Jira story (or an export of several) into a draft set of test scenarios and test cases for you to review.

#### Step 1 — Get the Jira content into a file (skip if just pasting text)

If you have a Jira export (CSV or Excel with multiple stories), save it anywhere in the project, e.g.:

```
C:\etl_testing\docs\jira\my_export.csv
```

#### Step 2 — Convert it to text (skip if pasting a single story directly)

Open a terminal at the project root and run:

```
cd C:\etl_testing
.venv\Scripts\python.exe scripts\extract_test_cases.py
"docs\jira\my_export.csv"
```

This creates a readable file at `C:\etl_testing\output\_extracted\my_export.md`.

#### Step 3 — Open the project in Cursor

File → Open Folder → select `C:\etl_testing`.

#### Step 4 — Give Cursor's Agent chat one of these prompts

*If pasting a single story directly:*

```
Draft test scenarios and test cases from this Jira story: [paste the story's title, description, and acceptance criteria here]
```

*If you extracted a file in Step 2:*

```
Draft test scenarios and test cases from @output/_extracted/my_export.md
```

**What happens next**

Cursor writes a draft to output/\_drafted/<name>.md — test cases grouped under scenarios, plus an “Assumptions & Open Questions” section at the top listing anything it guessed or wasn't sure about.

*Read that section carefully — it tells you exactly what to double-check before moving on.*

### Step 5 — Review the draft

Open the .md file, read it through, fix anything wrong, and answer the open questions (edit the file directly, or tell Cursor the missing detail and ask it to redo that part).

---

## SECTION 2

### Draft Test Cases → Test Suites & Scenarios

Uses the **test-case-to-suite** Cursor skill. This is the step right after Section 1 — turning a reviewed draft into something the app can actually run.

#### Step 1 — Make sure you're happy with the draft

From output/\_drafted/<name>.md (produced in Section 1).

#### Step 2 — In the same (or a new) Cursor chat, give this prompt

*Convert the test cases in @output/\_drafted/my\_export.md into scenarios and suites.*

#### What happens

- Creates/updates the test scenario in config\scenarios.yaml
- Creates one suite file per test case (or grouped by hop) under config\suites\\*.yaml
- Business-rule test cases (e.g. “Fund must be CSP/CEMOF/CAP”) get added to the right mapping's business\_rules in config\mappings\<name>.json
- Report-tab test cases get added to config\reports\<id>.json
- Cursor reports back a table: test case → file → any gaps it couldn't fill in

#### Step 3 — See it in the app

Start the app (node server.js in Testing Orchestrator App\ ) and open <http://localhost:3000/scenarios> — the new scenario and its test cases should already be there (no restart needed if the app was already running).

#### Step 4 — Try running it

Run it from the scenario page (**Run all**) or a single case from /suites → /runs/new, to confirm it actually works before treating it as final.

---

## SECTION 3

### Existing Test Case Sheet → Suites & Scenarios

Same skill as Section 2 — just a different starting point: you already have a manually-written test case Excel sheet, not something from Jira.

#### Step 1 — Put the Excel file anywhere in the project

```
C:\etl_testing\docs\Historical Load.xlsx
```

### Step 2 — Convert it to text

```
cd C:\etl_testing
.venv\Scripts\python.exe scripts\extract_test_cases.py "docs\Historical
Load.xlsx"
```

Writes C:\etl\_testing\output\\_extracted\Historical Load.md. You can also convert a whole folder at once: `scripts\extract_test_cases.py docs\` converts every .xlsx/.csv inside it.

### Step 3 — In Cursor, give this prompt

*Convert the test cases in @output/\_extracted/Historical Load.md into scenarios and suites.*

Same result as Section 2: scenario + suites + rules created, a conversion table reported back — review before running.

*Your Excel sheet's exact column layout (which column is the scenario, which is the check type, etc.) is your own layout — Cursor infers it. If it's unclear, or your sheet uses manual "Step 1 / Step 2 / Step 3" instructions instead of structured columns, it will ask rather than guess.*

---

## SECTION 4

# Where to Put Mapping Documents

This covers the Bronze/Silver/Gold Excel mapping sheets the app uses to show table/column mappings and business rules.

### Step 1 — Put the Excel workbook here

```
C:\etl_testing\config\mappings\<name>_mapping.xlsx
```

e.g. `config\mappings\globalcredit_mapping.xlsx` — there are already example files there showing the expected sheet structure (**Tables**, **Columns**, **BusinessRules**, **ReferentialIntegrity**).

### Step 2 — Convert it to the format the app actually reads

The app reads JSON, not the raw Excel:

```
cd C:\etl_testing
.venv\Scripts\python.exe -m etl_test.cli export-mapping
"config\mappings\<name>_mapping.xlsx"
```

Creates `config\mappings\<name>_mapping.json` right next to it.

### Step 3 — Check it in the app

Open `http://localhost:3000/mappings` — the new mapping with its tables/columns/rules should be visible.

## Important

From this point on, edit the mapping through the app UI, not the Excel file again. The .json file becomes the real source of truth; the app doesn't re-read the .xlsx afterward. If you get an updated Excel later, re-running export -mapping will overwrite the .json — so re-apply any in-app edits afterward if needed.

No template yet? Generate a blank one to fill in:

```
.venv\Scripts\python.exe samples\make_mapping_template.py
```

---

## SECTION 5

# Shared OneDrive Setup (No Teams Needed)

Makes everyone's test runs, scenarios, and alerts show up for the whole team. This version uses OneDrive directly — no Teams channel required.

## Step 1 — One person creates the shared folder

Pick one person (doesn't matter who) to do this once:

- Open **File Explorer** → go to your **OneDrive** folder (shows as “OneDrive - <Company>” in the sidebar)
- Create a new folder, name it clearly, e.g. **ETL Test Runs**

## Step 2 — Share that folder with your teammate

- Right-click the **ETL Test Runs** folder → **Share**
- Type your teammate's work email address (the one tied to their OneDrive/Microsoft account on their VM)
- Set permission to “**Can edit**” (not “Can view”) — they need to write their own subfolder into it
- Click **Send**

*If right-click → Share isn't available, go to onedrive.com, log in, find/create the folder there, and share it from the website instead — same result.*

## Step 3 — Teammate accepts it on their machine

- They get an email with a link (or find it under **Shared** → **Shared with me** in their OneDrive)
- Open it and click “**Add shortcut to OneDrive**” (sometimes labeled “Add to My Files”)
- The folder now appears inside their own OneDrive too — same files, referenced from their account, not a copy

## Step 4 — Find the real local path on each machine

Don't assume the path — check it directly on each machine:

- On your VM: File Explorer → OneDrive folder → right-click **ETL Test Runs** → **Properties** → copy the **Location** path
- Ask your teammate to do the same on their VM

```
You: C:\Users\<you>\OneDrive - <Company>\ETL Test Runs  
Teammate: C:\Users\<teammate>\OneDrive - <Company>\ETL Test Runs
```

*Paths differ per person — that's expected. Sometimes a shared-then-added folder lands under a slightly different subpath, which is why checking Properties is safer than assuming.*

## Step 5 — Configure the app on your VM

```
cd C:\etl_testing\Testing Orchestrator App
copy .env.local.example .env.local
```

Open `.env.local` in a text editor and add (using your Step 4 path):

```
TEAM_RUNS_ROOT=C:\Users\<you>\OneDrive - <Company>\ETL Test Runs
TEAM_MEMBER=<your-name>
```

*TEAM\_MEMBER is just how your runs get labeled to the team — pick something readable like your first name. If skipped, it defaults to your Windows username.*

## Step 6 — Teammate configures the same on their VM

Same idea, but their own path from Step 4 and their own name for `TEAM_MEMBER`.

## Step 7 — Restart the app

Run `node server.js` again, on both machines.

## Step 8 — Verify it worked

Run any test on your machine → open `http://localhost:3000/runs` → confirm a **Member** column appears with your name. Once your teammate runs something, it shows up in your `/runs` list too (and yours in theirs) — no manual copying, OneDrive syncs the files in the background.

## Reminder

This only shares **run history** (runs, scenarios, alerts) — not `config\` (suites, mappings, connections). If you want identical test setups on both machines, copy those files over separately.